

Chapter 14 — Parameter-Efficient Fine-Tuning

pig7selene

September 5, 2026

Abstract

Parameter-Efficient Fine-Tuning (PEFT) adapts a pretrained model while retaining a shared frozen base and training a comparatively small task-specific state. This chapter develops the central low-rank construction behind LoRA, then compares it with quantized-base adaptation in QLoRA, inserted Adapter modules, Prefix Tuning, and Prompt Tuning. The central distinction is not simply the number of trainable parameters: PEFT changes the update parameterization, optimizer state, artifact topology, and sometimes the inference path, while leaving important costs such as activations and the base-model footprint in place.

1 Introduction

Chapter 13 introduced Supervised Fine-Tuning (SFT) as a change in the conditional data distribution and loss mask used to adapt a Causal Language Model. Its compact treatment of LoRA established the basic low-rank update, but did not compare the larger family of Parameter-Efficient Fine-Tuning (PEFT) methods. This chapter supplies that comparison. It assumes the SFT objective, Chat Template, and assistant-target masking from Chapter 13, and treats PEFT as a choice about **which state is permitted to change** under that same objective.

Full fine-tuning makes a task-specific copy of the entire pretrained parameter vector θ and optimizes it to obtain θ' . This is expressive, but a separate task or customer can require a separate full checkpoint, its optimizer state during training, and a deployment decision about which complete model to load. PEFT instead keeps a base parameter set θ_0 frozen and introduces a much smaller trainable state φ :

$$\theta = \theta_0 \rightarrow (\theta_0, \varphi), \quad \nabla_{\theta_0} \mathcal{L} = 0, \quad \nabla_{\varphi} \mathcal{L} \neq 0. \quad (1)$$

The expression in Equation 1 is a training contract, not a claim that the base model disappears from memory. The frozen weights must still be stored and read for every forward pass. What PEFT primarily avoids is a gradient and optimizer-state allocation for those weights, while representing adaptation in a portable additional artifact. Whether this is sufficient depends on the task, the base model, the selected modules, and the available compute; PEFT is not a guarantee that a small update matches unrestricted fine-tuning.

2 Full Fine-Tuning, Trainable State, and Memory

It is useful to separate four quantities that are often compressed into the phrase “memory efficient.” Let P denote the number of base-model parameters, P_{φ} the number of trainable adaptation parameters, and let the b terms denote bytes per stored element under a declared precision policy. A schematic training-memory decomposition is

$$M_{\text{train}} \approx M_{\text{base}} + P_{\varphi}(b_{\varphi} + b_{\text{grad}} + b_{\text{opt}}) + M_{\text{act}} + M_{\text{temp}}. \quad (2)$$

For ordinary full fine-tuning, $P_{\varphi} = P$ and M_{base} may share storage with the trainable parameter copy. Under PEFT, M_{base} remains a substantial model-weight term, but $P_{\varphi} \ll P$ can greatly reduce gradient and optimizer-state memory. Chapter 7 explains why Adam-style moment states make this distinction material. Activation memory M_{act} is governed by batch shape, sequence length, layer implementation, checkpointing, and the backward graph; it need not fall in proportion to the number of trainable parameters. Likewise, inference must normally retain the whole base model. A small adapter artifact reduces per-task storage, not the memory required to load the shared model.

The distinction also separates three operational claims. **Trainable parameter count** determines how many adaptation values receive updates. **Training-state memory** adds their gradients and optimizer state, alongside activations and temporary buffers. **Inference memory** contains the loaded base model, any unmerged adaptation state, and request-dependent state such as the KV Cache described in Chapter 24. PEFT can improve all three in particular designs, but none follows automatically from the first.

3 Low-Rank Adaptation

Low-Rank Adaptation (LoRA) is the central PEFT construction because it changes an existing linear transformation without inserting a new hidden layer into the Transformer. For a frozen pretrained matrix $W \in R^{d_{\text{out}} \times d_{\text{in}}}$, LoRA learns a low-rank update

$$\Delta W = BA, \quad A \in R^{r \times d_{\text{in}}}, \quad B \in R^{d_{\text{out}} \times r}, \quad r \ll \min(d_{\text{in}}, d_{\text{out}}), \quad (3)$$

and uses the adapted transformation

$$W' = W + \frac{\alpha}{r}BA. \quad (4)$$

Here r is the adaptation rank and α is a scaling hyperparameter. The convention $\frac{\alpha}{r}$ is common, but an implementation must state its actual scale and whether it is folded into a factor. Since $\text{rank}(BA) \leq r$, Equation 3 restricts the update to a low-rank subspace. It does not assert that the ideal task-specific update has low rank in every setting; it is an empirically useful parameterization whose capacity increases with r [1].

The original matrix contains $d_{\text{out}}d_{\text{in}}$ parameters. The two LoRA factors contain

$$P_{\text{LoRA}} = r(d_{\text{in}} + d_{\text{out}}), \quad (5)$$

which can be much smaller when r is modest relative to both dimensions. For an input $x \in R^{d_{\text{in}}}$, the forward calculation is better understood as two paths:

$$W'x = Wx + \frac{\alpha}{r}B(Ax). \quad (6)$$

The frozen path Wx remains present during training. LoRA adds the narrow projection Ax and its expansion $B(Ax)$; it does not replace the pretrained matrix with a small matrix. This distinction explains both its modularity and why base-weight bandwidth still matters.

3.1 Rank, Initialization, and Target Modules

Rank governs a direct trade-off. A larger r enlarges the attainable update space and the trainable state in Equation 5; it also increases optimizer memory and low-rank-path arithmetic. A small r is cheaper and can regularize adaptation, but may underfit a task or prove inadequate for the chosen target modules. The useful rank is consequently an empirical decision tied to data size, task shift, selected projections, and evaluation, rather than a universal property of a model family.

A common initialization samples one factor, for example A , and initializes the other factor B to zero. Then $BA = 0$ at the first update and Equation 4 initially reproduces the base transformation exactly. This avoids an arbitrary perturbation of the pretrained function at initialization while still allowing nonzero gradients for the zero-initialized factor. Factor orientation, initialization distribution, scaling, dropout, and precision are all part of an adapter artifact's specification.

In a Transformer, LoRA can target the Query, Key, Value, and output projections in Self-Attention, or one or more expansion and contraction projections in the Feed-Forward Network. Chapter 3 defines these attention projections and Chapter 4 defines the feed-forward path. Selecting only Query and Value projections yields a different update budget and inductive bias from adapting every attention and MLP projection. There is no architecture-independent target set: attention-only adaptation can be sufficient for one distributional shift, while another task may require feed-forward capacity as well.

3.2 Merging and Multiple Adapters

For a compatible ordinary linear layer, the trained update can be folded into a copy of the base weight:

$$W_{\text{merged}} = W + \frac{\alpha}{r}BA. \quad (7)$$

Merging removes the extra low-rank path for that fixed adaptation at inference, but produces a task-specific full weight copy. Keeping the factors separate instead permits one frozen base model to coexist with many small adapters. These alternatives answer different operational needs:

Serving choice	Advantage	Consequence
Merged update	One fixed linear weight and no separate LoRA branch in the execution graph.	Consumes a task-specific full-weight copy; switching requires another merged state.
Separate adapter	One base can load or select many small adaptation artifacts.	Runtime must identify the base, adapter, scale, target modules, and compatible kernels.
Composed adapters	Can express a declared combination of adaptations.	Updates may interfere; order, weights, and evaluation must be explicit.

Table 1: Merging optimizes a fixed deployment path; separate adapters optimize sharing and selection.

Neither representation is intrinsically safer without a compatibility contract.

Storing an adapter, loading it into an execution process, switching a request to it, and composing several adapters are distinct operations. In particular, arithmetic addition of two low-rank updates is not evidence that their behaviors compose usefully. A server must associate every adapter with a specific base checkpoint, tokenizer and model configuration, factor orientation, scaling convention, target module map, and merge status.

4 QLoRA: Quantized Frozen Bases

QLoRA combines a quantized frozen base model with higher-precision trainable LoRA parameters. Its conceptual path is

$$\text{quantized base} + \text{LoRA factors} \rightarrow \text{adapted model}. \quad (8)$$

The quantization in Equation 8 is not ordinary inference quantization applied after training. The base weights remain quantized and frozen during adaptation; gradients pass through the computation needed to update the LoRA factors, not into an independently updated full-precision base. This reduces the weight-storage term in Equation 2 while preserving a trainable adaptation path. Dettmers et al. use this design to make substantially larger frozen bases practical under constrained accelerator memory [2].

Storage precision, computation precision, and trainable-parameter precision must remain separate. A four-bit representation can store a block of frozen weights compactly, while a kernel dequantizes or computes using a higher-precision representation, and the LoRA factors, gradients, and optimizer states may use another dtype. Thus “4-bit fine-tuning” does not mean every operation or every trainable quantity is four-bit.

4.1 NF4, Double Quantization, and Paged Optimizers

QLoRA introduced NormalFloat4 (NF4), a four-bit data type designed for weights with an approximately normal distribution, together with block-wise quantization. At the chapter’s level of abstraction, NF4 chooses a compact set of representable values better matched to that assumed weight distribution than a uniform integer grid; it is not a claim that every tensor is normally distributed or that NF4 is always the correct format. Chapter 25 develops the general distinction between quantization representations, scales, reconstruction error, and kernel support.

Each quantized block also needs quantization constants, such as scales. **Double quantization** compresses these constants themselves, reducing a secondary metadata cost. It should not be confused with applying two independent quantizers to the base weight: its purpose is to shrink the representation overhead surrounding block quantization.

Paged optimizers address a different term in the memory picture. Temporary optimizer or activation-related allocation spikes can exceed a device’s capacity even when steady-state accounting appears feasible. The QLoRA implementation uses a paging mechanism backed by unified memory to manage

such spikes. This can enlarge the set of feasible experiments, but page migration is not free: it can introduce transfer overhead and does not eliminate activation, sequence-length, or kernel constraints [2].

The resulting qualitative decomposition is

$$M_{\text{QLoRA}} \approx M_{\text{quantized base}} + M_{\text{LoRA factors, gradients, and optimizer}} + M_{\text{activations}} + M_{\text{temporary buffers}}. \quad (9)$$

The first term can fall sharply relative to a full-precision frozen base, but long sequences and large microbatches can still make the remaining terms decisive. QLoRA is therefore a memory-budgeting technique, not a substitute for checking the effective batch, activation policy, and numerical behavior described in Chapters 7 and 8.

5 Inserted and Prompt-Based Adaptation

LoRA parameterizes a modification of an existing linear map. Other PEFT methods place the small trainable state at different points in the computation. This changes what is learned, what must execute at inference, and how readily the state can be attached to a compatible base.

5.1 Adapters

An Adapter layer inserts a small trainable bottleneck into a Transformer block while leaving most pretrained parameters frozen. A representative residual Adapter maps a hidden vector $h \in R^d$ to

$$h' = h + W_{\text{up}}\varphi(W_{\text{down}}h), \quad W_{\text{down}} \in R^{m \times d}, \quad W_{\text{up}} \in R^{d \times m}, \quad (10)$$

where $m \ll d$ is the bottleneck width and φ is a nonlinearity. Like LoRA rank, m controls an efficiency–capacity trade-off. Adapters introduce new modules into the execution graph rather than changing an existing projection by a low-rank update. They are consequently compact and modular, but can add inference work and architectural integration requirements at every insertion point. Houlshy et al. established this frozen-backbone, bottleneck-module pattern for parameter-efficient transfer [3].

5.2 Prefix Tuning and Prompt Tuning

Prefix Tuning learns continuous task-specific states that condition attention while the language-model parameters remain frozen. In a decoder-only layer, the learned prefix can be viewed conceptually as additional Key/Value-like states available to later tokens. These vectors are not ordinary textual tokens with vocabulary IDs; they are trainable continuous states whose placement and layer-wise parameterization must be specified. Prefix Tuning therefore changes conditioning at attention interfaces rather than directly changing W in Equation 4 [4].

Prompt Tuning learns a smaller sequence of **soft prompt** embeddings prepended to the ordinary token embeddings. If $E_{\text{soft}} \in R^{p \times d}$ contains p learned prompt vectors and $E(x)$ is the embedding sequence for tokenized input x , the Transformer receives

$$[E_{\text{soft}}; E(x)]. \quad (11)$$

Textual prompts are discrete strings passed through the tokenizer; soft prompts are optimized vectors and need not correspond to readable vocabulary items. Prompt Tuning changes the input embedding sequence, whereas Prefix Tuning supplies learned conditioning states to attention layers. Both preserve the frozen base weights, but they are not synonyms and have different capacity, context-budget, and implementation behavior [5].

6 Comparing PEFT Methods

The methods differ along more than one axis. The following table is deliberately qualitative: trainable share, quality, and throughput depend on model scale, target modules, task shift, sequence length, and implementation.

Method	Trainable state	Base or graph change	Typical inference implication	Primary trade-off
Full fine-tuning	All or nearly all model parameters	Updates base weights	One full adapted checkpoint	Maximum update freedom; highest trainable-state cost.
LoRA	Low-rank factors on selected projections	Frozen base; additive update	Can merge or select separate adapters	Target and rank determine capacity.
QLoRA	LoRA factors with quantized frozen base	Quantized base during adaptation	Quantization-compatible serving path	Reduces base storage; adds quantization constraints.
Adapters	Bottleneck modules	Adds modules to Transformer blocks	Extra module execution unless fused	Modular capacity with architectural overhead.
Prefix Tuning	Layer-level continuous prefixes	Conditions attention states	Prefix state participates in execution	Small state; per-layer conditioning design.
Prompt Tuning	Input-level soft prompt	Extends embedding sequence	Consumes context positions	Very small state; capacity may be limited.

Table 2: PEFT methods choose different locations for task-specific state. The table compares mechanisms, not guaranteed quality rankings.

For a deployment that serves many narrowly differentiated tasks, separate small adapters can reduce artifact storage and enable explicit task selection. For one fixed production task, merging a LoRA update can simplify the execution path. For memory-constrained training, a quantized frozen base can be more important than reducing the adapter size further. Conversely, a method with few trainable parameters can underperform unrestricted fine-tuning when the required behavioral shift is broad, the prompt or prefix capacity is too limited, or the selected LoRA modules omit the computation that must change.

PEFT does not relax the data and evaluation obligations of SFT. A low-rank update can still overfit duplicated instruction data, memorize stylistic artifacts, or cause capability regression. The protected evaluation and compatibility checks of Chapter 13 remain necessary; the reduced trainable state changes the adaptation interface, not the definition of a trustworthy result.

7 Failure Modes

Several failure modes arise from treating a PEFT artifact as merely a small tensor file. A rank that is too small can leave an adaptation under-capacitated; a needlessly large rank can erase its resource advantage and overfit limited data. A poor target-module choice can create the same symptom even with a large rank, because the required transformation is inaccessible. Unstable learning rates, inappropriate scaling, or incorrectly frozen base weights can make a supposedly small update destructive. QLoRA adds representation-specific risks. Poorly calibrated or unsupported low-precision kernels can remove the expected memory or speed benefit, and quantization error can interact with a task-sensitive layer or distribution. A reported “quantized” configuration is incomplete unless it identifies the base checkpoint, quantization scheme and block rules, compute dtype, adapter dtype, and kernel implementation. Activation pressure can still cause out-of-memory failures even when the weight footprint fits.

At serving time, the most serious errors are often compatibility errors: attaching an adapter to the wrong base revision, mixing factor orientations or scaling conventions, merging the wrong update twice, or composing adapters whose joint behavior was never evaluated. A correct deployment therefore treats the adapter identifier and its declared base-model contract as inseparable.

8 Implementation Contracts

The **base-model contract** must identify the exact checkpoint, architecture configuration, tokenizer and Chat Template where the adaptation is conversational, weight dtype, and target-module name map. It must state which base parameters are frozen and include a test that their values and gradients remain unchanged after an optimization step.

The **adaptation contract** must identify the PEFT family, trainable parameter count, factor orientation, rank or bottleneck width, scaling, initialization, dropout, target modules, and any merge state. For QLoRA it must additionally record the quantization format, block policy, quantization-constant representation, computation dtype, and optimizer paging policy. A shape test should verify every A , B , Adapter, prefix, or soft-prompt tensor against its declared model interface before loading weights. The **optimization contract** retains the SFT data, masking, optimizer, precision, effective batch, sequence-length, gradient-accumulation, and checkpoint conventions from Chapters 7, 8, and 13. It should report both the number of trainable parameters and the observed peak memory by category where possible; reporting only adapter size can hide an activation or temporary-buffer bottleneck. Finally, the **serving contract** must specify whether updates are merged, separately selected, or composed; the adapter revision; all compatibility metadata; and the evaluation used to validate the exact assembled model. Tests should reject a mismatched base identifier, an unknown target module, inconsistent factor shapes, duplicate merging, and an adapter that changes a frozen base artifact on disk.

9 Summary

PEFT adapts a pretrained language model by preserving a shared frozen base and training a smaller task-specific state. Its principal savings arise in trainable parameters, gradients, optimizer state, and per-task artifact storage; base-weight, activation, and request-state memory remain independent accounting terms. Full fine-tuning and PEFT therefore differ in parameterization and operational topology, not merely in the number printed beside a model.

LoRA expresses an update as scaled low-rank factors and can target selected attention or feed-forward projections. QLoRA combines this update path with a quantized frozen base, using NF4, compressed quantization metadata, and paging techniques to improve constrained-memory training. Adapters insert bottleneck modules, while Prefix Tuning and Prompt Tuning learn continuous conditioning states at different locations in the Transformer computation.

The appropriate method follows from a declared task, memory budget, serving model, and evaluation plan. Rank, target modules, quantization policy, merge status, and base-model identity are all executable compatibility conditions. A PEFT artifact is useful only when those conditions preserve both the intended base model and the intended adaptation behavior.

References

- [1] E. J. Hu *et al.*, “LoRA: Low-Rank Adaptation of Large Language Models,” in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=nZeVKeeFYf9>
- [2] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” *arXiv preprint arXiv:2305.14314*, 2023, doi: 10.48550/arXiv.2305.14314.
- [3] N. Hounsby *et al.*, “Parameter-Efficient Transfer Learning for NLP,” in *Proceedings of the 36th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 2790–2799. [Online]. Available: <https://proceedings.mlr.press/v97/hounsby19a.html>
- [4] X. L. Li and P. Liang, “Prefix-Tuning: Optimizing Continuous Prompts for Generation,” in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Association for Computational Linguistics, 2021, pp. 4582–4597. doi: 10.18653/v1/2021.acl-long.353.
- [5] B. Lester, R. Al-Rfou, and N. Constant, “The Power of Scale for Parameter-Efficient Prompt Tuning,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2021, pp. 3045–3059. doi: 10.18653/2021.emnlp-main.243.